Nonexistence of Williamson matrices of order 65

P. Graham
https://orcid.org/0009-0007-8493-1505

October 22, 2026

Abstract

Williamson matrices of order n are four symmetric circulant sign matrices whose squares sum to $4nI_n$. Their construction dates to Williamson's 1944 work on Hadamard matrices. We give a computer-assisted nonexistence result at order 65, described by Bright in 2019 as "currently the smallest order for which it is unknown" whether Williamson matrices exist [14, Section 4.2]. Order 67 remains open. The search uses row sums, spectral inequalities, and compression to reduce the problem, then exhaustively matches pairs of full rows. Recorded case and pair totals are checked separately from the search. The conclusion also depends on correct execution of the pair matching, which has not been formally verified.

1 Introduction

A *sign matrix* has entries in $\{1, -1\}$. A matrix is circulant when each row is the cyclic shift of its predecessor. Four symmetric circulant sign matrices A, B, C, D of order n are *Williamson matrices* if

$$A^2 + B^2 + C^2 + D^2 = 4nI_n, \tag{1}$$

where I_n is the identity matrix. We establish the following computer-assisted result.

Theorem 1. *There are no four symmetric circulant sign matrices A, B, C, D of order 65 satisfying $A^2 + B^2 + C^2 + D^2 = 260I_{65}$.*

This is a statement about symmetric circulant blocks, not about arbitrary Hadamard matrices of order 260 or the broader Williamson-type families.

Exhaustive computer search with a replayable CPU implementation, receipts and two independent coverage and accounting checkers.
Computer-assisted; not yet reviewed by experts. The proofs, the search code, the certificate checkers and the text were produced by AI models (from OpenAI and Anthropic), working under the author's direction.

1.1 History and the new case

We recall the existence question and explain how the present computation extends earlier classifications. Williamson introduced the construction in 1944 [1]. The odd-order Williamson conjecture asserts that these four blocks exist for every odd positive integer n; Turyn recorded the more general all-orders conjecture in 1972 [2]. Baumert, Golomb and Hall's computer construction at block order 23 yielded a Hadamard matrix of order 92 [3]. Baumert and Hall subsequently enumerated the odd block orders through 23 [4]. Later searches by Sawade, Koukouvinos and Kounias, and

Đoković [5, 6] extended the odd-order classifications; the history, including corrections to incomplete enumerations, is reviewed by Bright, Kotsireas and Ganesh [12, Section 1]. Đoković disproved the conjecture at order 35 in 1993 [7]. Holzmann, Kharaghani and Tayfeh-Rezaie completed the odd-order enumeration through 59, including nonexistence at 47, 53 and 59 [8]. Their work dates to a 2007 manuscript [8]. Turyn's construction supplies orders 61 and 63. Lang and Schneider also classified order 61, as credited by Bright, Kotsireas and Ganesh [12, Section 1], in their work on Turyn-type Williamson matrices [9]. Bright, Kotsireas and Ganesh's MathCheck SAT and computer-algebra work, first posted in 2018, enumerated all even orders through 70 and the odd multiples of three through 69, including a new class at order 63 [12]. Bright's March 20, 2019 research statement describes 65 as "currently the smallest order for which it is unknown" whether Williamson matrices exist [14, Section 4.2]; it also lists 67 as unenumerated. The present theorem answers the order-65 case negatively; order 67 remains open. The Wikipedia article on the Williamson conjecture lists nonexistence at $35, 47, 53, 59$; it does not identify 65 as open.[1] The open status of 65 cited here rests on Bright's 2019 statement and version 2 (September 6, 2026) of Kharaghani, Mohammadian, Tayfeh-Rezaie and Zaitsev's paper [16, Section 2.3].

There is substantial recent progress for a related family. Kharaghani, Mohammadian, Tayfeh-Rezaie and Zaitsev's version 2 [16] constructs *near-Williamson* matrices at order 65: these satisfy $AA^{\mathsf{T}}+BB^{\mathsf{T}}+CC^{\mathsf{T}}+DD^{\mathsf{T}} = 260I_{65}$, with B, C, D symmetric, while A need not be symmetric. The work was first posted on May 9, 2026 as a three-author version 1 by Kharaghani, Mohammadian and Tayfeh-Rezaie, titled *A search for Hadamard matrices of Williamson type*, with existence through odd order 63. The four-author version 2, dated September 6, 2026, establishes near-Williamson existence at every odd order through 69 and enumerates that broader family through odd order 47. Its order-65 example does not have symmetric A. Our theorem excludes only the construction with four symmetric circulant blocks. Section 2.3 of version 2 [16] still includes 65 among the Williamson orders for which matrices have not been found; its order-65 search constructs near-Williamson examples and does not exhaust the symmetric case. The recent survey by Egan, Kotsireas, London and Ó Catháin, first posted on October 19, 2026, places these developments in the broader Hadamard construction problem [15]; it records the known nonexistence orders $35, 47, 53, 59$, without reporting a settlement of the order-65 case.

Bennett, Bright, Colinot and Nayak's work, first posted January 29, 2026, enumerates circulant Williamson-type matrices through order 21, allowing nonsymmetric blocks [13]. Their correspondence with quaternionic perfect sequences gives an enumeration algorithm for that broader family. This is relevant progress in Williamson-type enumeration; the present order-65 search concerns symmetric blocks and uses neither their code nor their quaternionic correspondence.

Zhang's April 2, 2026 report [10, Section 7.4] describes an AI-assisted attempt on the exact order-65 problem and expressly leaves it unsolved. This earlier attempt is relevant background; none of its code, data, or proposed reductions is used here.

The new contribution is completion of the order-65 problem with no restriction on the four symmetric circulant blocks beyond their defining equation. Compression and spectral restrictions are established methods, developed in particular by Đoković and Kotsireas [11] and used in the MathCheck SAT and computer-algebra enumeration [12]. The difficult step here is excluding *every* lift of a possible compression, meaning every choice of full rows having that compression. Excluding the compressed objects alone does not settle the question. A direct join of pairs of full rows, with conservative spectral bounds, completes this step. This replaces the SAT-based lifting step of Bright, Kotsireas and Ganesh's MathCheck system, whose published enumeration treats even orders and orders divisible by three. Their enumeration does not include 65, which has neither of those

[1] https://en.wikipedia.org/wiki/Williamson_conjecture, consulted October 22, 2026.

properties. Holzmann, Kharaghani and Tayfeh-Rezaie already used row and pair spectral bounds [8, Section 3]. After choosing the first two blocks, they used the product identity to eliminate the fourth and solved linearized equations modulo 4 for the third, checking the original equations finally. Here compression, a multiplier restriction on one row, and two-sided pair bounds instead precede a direct join of the two full-row pairs. These are combinations of established tools, not a new general construction principle. The measured cost below describes this implementation; no controlled comparison with the earlier programs is available, and no speedup over them is claimed. The central limitation is explicit: checking the records is much cheaper than repeating the search, but does not prove correctness of the filtering and matching operations.

Section 2 gives the argument's structure. Sections 3–5 justify the finite reductions and matching test. Section 6 states the computational outcome, controls, replay instructions, and precise trust boundary.

2 Proof overview

We first describe how the matrix equation becomes a finite enumeration. A symmetric circulant matrix of odd order is determined by half of its first row and its initial entry. Negating a whole matrix leaves its square unchanged, so each initial entry may be fixed to $+1$. At order 65 this leaves a 32-bit mask for each row. Row sums and Fourier eigenvalues give necessary conditions that can be checked separately for each row.

Next, sum entries in each residue class modulo 13. This compresses a row of length 65 to an integer vector of length 13. Partition the retained rows into classes according to their row sum and compression. Every solution determines an admissible tuple: four classes satisfying the exact compressed autocorrelation equations. A lift of that tuple is a choice of one full row from each class. Common multiplication of the indices by an invertible residue gives equivalent tuples, so only one tuple from each orbit is needed. A search unit is one such representative. This is a cover of the full problem because every solution can be carried to one of the representatives by the permitted symmetries. Multipliers that fix the compression allow a further restriction on one designated row, whose mask may be chosen least among its equivalent masks.

Within a unit, select one of the three ways of splitting its four row lists into two pairs. Spectral bounds reject pairs that cannot be completed by the opposite two lists. These bounds use outward integer enclosures of the squared eigenvalues; the error margin ensures that numerical rounding cannot exclude a solution. The surviving pairs are joined using necessary autocorrelation and product identities. Every matching candidate is tested by exact integer arithmetic. Repeated classes and repeated rows are allowed throughout.

Finally, the absence of solutions in every unit, combined with the cover, implies Theorem 1. This last step uses the correctness of the executed search. The computation and its trust assumptions are described separately in Section 6.

3 Rows, autocorrelations, and spectra

This section fixes the encoding and proves the necessary conditions used to retain individual rows. All indices below are taken modulo the odd order n, and $H = (n-1)/2$.

For example, at order 5 a normalized symmetric first row has the form $(1, a, b, b, a)$, with $a, b \in \{1, -1\}$. In general a normalized row is a sequence $x = (x_i)_{i \in \mathbb{Z}/n\mathbb{Z}}$ with $x_0 = 1$ and $x_i = x_{-i}$. Encode it by the H-bit integer m_x whose bit $i-1$ is one exactly when $x_i = -1$, for $1 \le i \le H$. The

function popcount(m_x) counts the one bits of this mask. Define its periodic autocorrelation by

$$\mathrm{PAF}_x(s) = \sum_{i \in \mathbb{Z}/n\mathbb{Z}} x_i x_{i+s}.$$

For a symmetric circulant matrix with first row x, the entry in position $(0, s)$ of its square is $\mathrm{PAF}_x(s)$. Thus the matrix equation is exactly

$$\mathrm{PAF}_a(s) + \mathrm{PAF}_b(s) + \mathrm{PAF}_c(s) + \mathrm{PAF}_d(s) = \begin{cases} 4n, & s = 0, \\ 0, & s \neq 0. \end{cases} \tag{2}$$

Whenever a sum below runs over the four rows, it counts their four positions, including repetitions. Autocorrelation is symmetric in s, so testing $0 \leq s \leq H$ suffices.

Lemma 2. *If Williamson matrices of odd order n exist, their first rows may be normalized to $x_0 = 1$. Each normalized row has row sum $r_x = n - 4\,\mathrm{popcount}(m_x)$, and the four row sums satisfy $r_a^2 + r_b^2 + r_c^2 + r_d^2 = 4n$.*

Proof. Negation preserves every autocorrelation. Negative entries in a normalized symmetric row occur in pairs, giving the stated row sum. Summing (2) over all shifts gives the sum of the squares of the row sums. □

At order 65 the nondecreasing lists of absolute row sums are exactly

$$(1, 3, 5, 15),\ (1, 3, 9, 13),\ (3, 3, 11, 11),\ (3, 7, 9, 11),\ (7, 7, 9, 9).$$

Each sign is forced by $r_x \equiv 65 \pmod 4$, giving the allowed signed values $1, -3, 5, -7, 9, -11, 13, -15$. Permuting the four rows therefore permits sorting by the absolute row sums without loss.

For frequency $k \in \mathbb{Z}/n\mathbb{Z}$, the real eigenvalue of the circulant is

$$\lambda_x(k) = 1 + 2\sum_{i=1}^{H} x_i \cos(2\pi i k/n).$$

The Fourier transform diagonalizes all circulant matrices. Consequently (2) is equivalent to

$$\lambda_a(k)^2 + \lambda_b(k)^2 + \lambda_c(k)^2 + \lambda_d(k)^2 = 4n \qquad (k \in \mathbb{Z}/n\mathbb{Z}). \tag{3}$$

In particular every solution row satisfies $\lambda_x(k)^2 \leq 4n$. The row generator considers every mask and retains only permitted row sums and rows satisfying this necessary spectral bound, with the numerical slack justified in Section 5.1.

4 Compression and complete coverage

We now divide the possible solutions into finitely many units and justify both symmetry reductions. These are reductions of the search domain; they do not assert that a compressed solution necessarily lifts.

For $n = qL$, with compression factor q, define the compression of a row x by

$$X_j = \sum_{t=0}^{q-1} x_{j+Lt}, \qquad j \in \mathbb{Z}/L\mathbb{Z}.$$

Here $q = 5$ and $L = 13$. A class consists of the retained rows with the same signed row sum and compressed vector. Write $\mathrm{PAF}_X(s) = \sum_{j \in \mathbb{Z}/L\mathbb{Z}} X_j X_{j+s}$ for compressed autocorrelation.

Lemma 3. *For four rows satisfying (2), their compressions satisfy*

$$\mathrm{PAF}_A(s) + \mathrm{PAF}_B(s) + \mathrm{PAF}_C(s) + \mathrm{PAF}_D(s) = \begin{cases} 4n, & s = 0 \pmod{L}, \\ 0, & s \neq 0 \pmod{L}. \end{cases}$$

Proof. Expanding the compressed autocorrelation gives $\mathrm{PAF}_X(s) = \sum_{t=0}^{q-1} \mathrm{PAF}_x(s + Lt)$. For a nonzero residue s none of these shifts is zero modulo n. For $s = 0$ exactly one is zero. Summing over the four rows proves the claim. □

Call a tuple of four classes *admissible* when its compressed vectors satisfy Lemma 3 and its row sums satisfy Lemma 2. A *lift* is a tuple of four retained rows, one from each class; it need not itself solve the matrix equation. Classes are ordered lexicographically by the seven entries $(X_0, \ldots, X_6)$ of their symmetric compression. A class identifier is its zero-based rank in this order; the data files use those identifiers. A *sorted* class tuple has nondecreasing absolute row sums and, within equal row sums, nondecreasing class identifiers. Acting by a multiplier means applying it to each compressed vector and then re-sorting within equal-row-sum positions. The generator and cover check use this same order.

The admissible class tuples are enumerated by matching sums of two exact compressed autocorrelation vectors to the complementary pair sums. Every ordered class pair is included; equal classes are permitted. Sorting positions of equal absolute row sum removes only row permutations. The complete tuple set is also enumerated independently by matching the exact integer vectors themselves.

A *multiplier* is an invertible residue u modulo 65, acting on a row by $x_i \mapsto x_{ui}$. It preserves (2), since it permutes the shifts. Its action on the compression depends only on its residue modulo 13. Opposite multipliers act identically on symmetric rows. Every invertible residue modulo 13 lifts to an invertible residue modulo 65: among $u, u + 13, \ldots, u + 52$, exactly one is divisible by 5, so the other four are coprime to 65.

Lemma 4. *Let one representative of each orbit of admissible sorted class tuples under* $(\mathbb{Z}/13\mathbb{Z})^*/\{1, -1\}$ *be retained. Every order-*65 *Williamson solution is equivalent, by a common multiplier and a permutation of its rows, to a lift of a retained tuple.*

Proof. A solution's tuple is admissible by Lemma 3. Choose a multiplier carrying it to its orbit representative, lift that multiplier modulo 65, and apply it to all four full rows. Re-sort rows of equal absolute row sum. The resulting solution has the representative tuple. All lifts of the representative are searched, even if more than one multiplier carries the original tuple to it. Multipliers permute the exact eigenvalues. Transformed solution rows therefore still satisfy $\lambda_x(k)^2 \leq 260$ and pass the numerical filter with its proved margin. □

A *unit* is one representative in this lemma. The cover check computes the exact compressed tuples and verifies membership of their representatives in the supplied unit list. It checks invalid and duplicate representatives as well as missing ones.

A further symmetry reduces one row list in each unit. The subgroup

$$K = \{u \in (\mathbb{Z}/65\mathbb{Z})^* : u \equiv 1 \pmod{13}\} = \{1, 14, 27, 53\}$$

fixes every compressed vector. Choose one designated position in the unit and retain only rows whose mask is least in its K-orbit. Applying a common element of K that minimizes that row preserves a solution in the same unit. It is essential that this restriction applies to only one designated row; independent minimization of all four rows would not be justified.

5 Filtering and matching full rows

This section shows that every solution lifting a unit survives the pair filters and is reached by the join. A pairing is one of the three splits $(a, b) \mid (c, d)$, $(a, c) \mid (b, d)$, or $(a, d) \mid (b, c)$. The implementation may choose any of these pairings and any designated row; these choices affect cost only.

5.1 Conservative numerical enclosures

We use integer enclosures to make the spectral comparisons safe under rounding. At order 65, the error in each computed squared eigenvalue is below $4 \cdot 10^{-11}$ under the arithmetic assumptions and cosine-table checks detailed in Appendix A. The row filter rejects only when the computed square exceeds $260 + 10^{-6}$; it therefore retains every solution row.

Let $\widetilde{\lambda}_x(k)$ be the computed eigenvalue. Define integer lower and upper enclosures

$$\ell_x(k) = \max\{0, \lfloor 64\widetilde{\lambda}_x(k)^2 - 0.01 \rfloor\}, \qquad h_x(k) = \lceil 64\widetilde{\lambda}_x(k)^2 + 0.01 \rceil.$$

The error bound, including the two additional floating-point additions, implies

$$\ell_x(k) \le 64\lambda_x(k)^2 \le h_x(k). \tag{4}$$

Clamping the lower bound to zero preserves this enclosure, since a square is nonnegative. No strictly positive gap is claimed after clamping.

5.2 The pair filter

We now use the opposite pair of row lists to bound the possible spectral contribution of each pair. Consider two lists $\mathcal{A}, \mathcal{B}$ on one side and $\mathcal{C}, \mathcal{D}$ on the other, after restricting the designated row. Set $Q = 64 \cdot 260$. For each frequency let

$$U(k) = \max\{0, Q - \min_{c \in \mathcal{C}} \ell_c(k) - \min_{d \in \mathcal{D}} \ell_d(k)\},$$

$$V(k) = \max\{0, Q - \max_{c \in \mathcal{C}} h_c(k) - \max_{d \in \mathcal{D}} h_d(k)\}.$$

Retain a pair (a, b) only when, at every frequency,

$$\ell_a(k) + \ell_b(k) \le U(k), \qquad h_a(k) + h_b(k) \ge V(k). \tag{5}$$

The opposite side uses the same rule with the two sides interchanged.

Lemma 5. *Every pair belonging to a solution in the four lists passes (5) on both sides.*

Proof. For its complementary rows c, d, equation (3) gives $64(\lambda_a(k)^2 + \lambda_b(k)^2) = Q - 64\lambda_c(k)^2 - 64\lambda_d(k)^2$. Replacing each square by the appropriate lower or upper enclosure proves the inequalities before clamping. Raising a negative upper bound to zero only weakens the upper test. Raising a negative lower bound to zero changes nothing, because $h_a + h_b$ is nonnegative. □

If a list is empty there is no solution in the unit. The implementation details are given in Appendix B.

5.3 The product identity and the join

The second matching condition is Williamson's classical product identity [1, 12]. We include its short argument to fix the normalization.

Lemma 6. *For an odd-order Williamson solution normalized by $x_0 = 1$ in each row, $a_i b_i c_i d_i = -1$ for every nonzero index i. Equivalently, $m_a \oplus m_b \oplus m_c \oplus m_d = 2^H - 1$, where $\oplus$ is bitwise exclusive-or on H bits.*

Proof. In the integral group ring of the cyclic group of order n, write J for the sum of all group elements and T_x for the sum of the positions where row x is negative. Then $x = J - 2T_x$ and $x^2 = nJ - 4|T_x|J + 4T_x^2$. Symmetry and normalization make $|T_x|$ even. Modulo 2, the square T_x^2 equals the sum of the squares of its individual group elements, because mixed terms occur twice. Reducing $\sum_x x^2 = 4n\,1$ modulo 8 and dividing by 4 modulo 2 gives

$$\sum_x \sum_{t \in T_x} t^2 = 1 + J \pmod 2.$$

Squaring permutes a cyclic group of odd order. Every nonidentity position therefore occurs in an odd number of the four negative sets, proving the product identity and its mask form. □

For each row, form a linear hash of its nonzero-shift integer autocorrelations,

$$f(x) = \sum_{s=1}^{H} R_s \,\mathrm{PAF}_x(s) \pmod{2^{64}},$$

using fixed odd coefficients R_s; their reproducible choice is described in Appendix B. A solution satisfies $f(a) + f(b) = -f(c) - f(d)$ and $m_a \oplus m_b = (2^H - 1) \oplus m_c \oplus m_d$. The search combines these two necessary equalities into a single 64-bit key, using the same function on both sides. Specifically it uses $\alpha \oplus R\beta$ for hash sum α and mask value β, with a fixed odd R, and maps a zero key to one. The right side uses the negative hash sum and the H-bit complement of its mask XOR. Equal inputs give equal keys; hash collisions may add candidates but cannot remove a solution.

The join retains every pair with an equal key, including duplicates. Its storage and restart rules, justified in Appendix B, do not remove any such pair.

Finally, each key match is checked using the four exact integer autocorrelation vectors at shifts $0, \ldots, H$. Equation (2) then decides the matrix equation exactly. A separate witness checker can also form the four matrices and multiply them directly. This last check prevents false positives; it cannot repair a missing pair earlier in the search.

6 Computational result and reproducibility

This section states the finite outcome on which the theorem depends and separates the checks of its records from re-execution of the search. The programs and small data accompany the paper in `checks/` and `data/`.

6.1 The exhaustion record

We state the finite computational result needed to finish the proof. The CPU search regenerates its inputs from the problem definition. Its counts are shown in Table 1. A receipt is a record for one unit: the four classes, pairing, designated row, list sizes, enumerated pair counts, survivor counts,

matching keys, solutions, and number of passes. A pass processes one part of the pair lists, as defined in Appendix B; every order-65 unit uses one pass. A key match is a surviving left pair and a surviving right pair with equal keys. A surviving pair is counted once for each side of each unit where it occurs. The complete receipt collection has no unresolved unit.

Quantity	Count
Normalized masks examined	2^{32}
Retained rows	396,050,997
Compressed classes	15,251
Admissible sorted class tuples	423,527
Units after multiplier reduction	70,699
Enumerated pairs, both sides	70,830,746,132,488
Surviving pairs, both sides	1,645,891,558,124
Matches between pair keys	0
Solutions	0

Table 1: Counts in the complete CPU exhaustion record at order 65.

Computer-assisted conclusion of Theorem 1. Every solution may be normalized and belongs to the retained row set by Lemma 2 and the proved spectral margin. Lemmas 3 and 4 place an equivalent solution in one of the covered units. The restriction on the designated row preserves a solution in that unit. Lemmas 5 and 6 show that its two pairs survive and have complementary mask values. Moreover, $f(a) + f(b) + f(c) + f(d) = \sum_{s=1}^{H} R_s(\mathrm{PAF}_a(s) + \mathrm{PAF}_b(s) + \mathrm{PAF}_c(s) + \mathrm{PAF}_d(s)) = 0$ modulo 2^{64}, so their join keys are equal. The exhaustive joins report no key match in any unit. Subject to the search-execution trust base below, no solution exists. □

The complete CPU enumeration and two separately written complete GPU searches all find no solution. They use the same mathematical compression and symmetry reductions. The CPU implementation supplies its own row generator, cosine calculations, class and unit enumeration, and pair join; it does not read the GPU implementations' generated input files. The row sets and class counts agree. Their pair filters differ, so the CPU and GPU survivor counts are not expected to coincide and are not claimed to do so. Appendix G documents the prior GPU computations and the limits of their retained evidence. The per-unit comparisons below are between the CPU search and the NumPy join, not between the GPU searches. The public CPU implementation supplies a full replay without a GPU.

6.2 Two independent record checkers

The record checks establish the cover and its arithmetic accounting. The first regenerates CPU inputs using the CPU generators, but checks the cover with separately written code. The second has its own row, cosine, cover, and record-checking code written from the mathematical specification. Their row sets agree byte for byte, and both accept the complete cover and pair totals. Neither rederives survivor or key-match counts. Appendix F describes their scope and input sharing.

Here and throughout, independent means separately implemented, not produced by separate human teams or independently replicated by outside institutions. The AI provenance is stated on the first page.

Both checkers reject deliberately altered records and an incomplete cover. Appendix C states the changes tested.

These are *coverage and accounting* checks. Neither checker proves that every pair was correctly filtered, stored, and matched by the search. In particular, a search bug could lose all genuine matches while leaving consistent pair counts. The receipts are an exhaustion record, not a short independently checkable proof of all the unsuccessful joins.

6.3 Known cases and independent joins

Known solutions and independently computed pair joins test that the search can both recover solutions and reject false candidates. A *baseline enumeration* means a reference search with fewer reductions. The baseline used here, without the search-time spectral, compression, or product reductions, reproduces the published equivalence-class counts at every odd order from 3 through 33 [8, Table 1]:

n	3	5	7	9	11	13	15	17
#	1	1	2	3	1	4	4	4
n	19	21	23	25	27	29	31	33
#	6	7	1	10	6	1	2	5

The equivalences used for odd-order class comparison are row negation, row permutation, and a common invertible multiplier, as in [12, Section 2.2]. The trivial order 1 has one class as well.

Controls at orders 9 through 35 compare the reduced search with the baseline, switching individual reductions off, removing both row sorting and orbit reduction, and using compression alone. The corresponding solution sets agree after taking their *symmetry closures*: all normalized quadruples obtainable by permuting rows and applying a common invertible multiplier. The order-35 reference uses a spectral row filter and yields no solution.

A *row-filtered baseline* additionally applies the single-row spectral restriction, but no pair filter or product-key restriction. At order 39 the controls recover the single published class and compare with a row-filtered baseline. At order 45 multiple compression factors recover its single class; the variant comparisons use the symmetry closure of the search output as reference, so those comparisons are consistency checks, not an independent classification. At order 55, both compression factors 5 and 11 discover the single published class without receiving a witness as input. In particular factor 5 tests the same compression factor as order 65. These positive controls show that the search can recover solutions, including repeated rows, where they exist.

A separately written implementation using NumPy arrays reconstructs row lists, filters, and joins without sharing search code or using the product identity. It compares the list sizes and survivor counts and performs its own exact autocorrelation join. It agrees on all 3,309 order-45 units and recovers that order's solution.

At order 65, it checks 300 units sampled uniformly without replacement with NumPy's `default_rng(65)`, together with the 12 units having the largest enumerated-pair counts. These two sets are disjoint. The 312 units contain 7,955,197,270 surviving-pair instances, counted as above, about 0.4833% of the total in Table 1. Every sampled join gives zero solutions, and all list sizes agree with the receipts.

Both survivor counts agree with the records on every checked unit. The bookkeeping for numerical rounding boundaries, and further order-55 checks, is given in Appendix F. These are sample-based implementation tests, not a substitute for the complete enumeration. A positive matching test on order-65 data, described in Appendix D, also checks that deliberately shortened keys produce the same nonzero match count in separate join implementations.

6.4 Public replay and data

The public commands distinguish checking the record from repeating the complete search. From the public folder, install `checks/requirements.txt` and invoke `python3 checks/check.py`; the full requirements and mode descriptions are in `checks/README.md`. The default check regenerates the order-65 rows and cover with both checkers and validates all receipts and corruption controls. It also checks known-order counts, witness validity at orders 39, 45, and 55, and discovery at 39 and 45. It does not repeat all order-65 pair joins. The `-full-controls` option adds the unfiltered baseline counts through 33 and discovery at 55; the `-full-search` option repeats every order-65 CPU join. The default mode checks the supplied records; the last mode recomputes their mathematical results.

The full retained mask array alone has 1,584,203,988 bytes, one little-endian unsigned 32-bit integer per retained row. An optional dataset deposit could hold the complete row data, class and unit data, full receipt collection, and control outputs for comparison. The compact class, unit, and receipt data needed by the default check are already included in the public folder; the dataset is not required for that check. The optional deposit contents and replay commands are specified in `data/README.md`. The row data are regenerated from the public programs; no external dataset is needed to run them. No separate dataset deposit exists and no dataset DOI has been assigned. The complete CPU search recorded about 271 CPU-hours for its joins; this is a scale estimate for a replay, not the cost of the short record checks.

6.5 Trust boundary

The theorem depends on the written reductions, the completeness of the row and cover enumerations, and correct execution of the exhaustive search. It also depends on the compiler, hardware, integer arithmetic, floating-point model, and interval calculations used for the numerical margins. Independent implementations, corruption rejection, and positive controls supply evidence against implementation errors, but the search has not been formally verified. No claim of a Lean proof or a short standard unsatisfiability certificate is made. A reader seeking to repeat the decisive computation must execute the full search, not only check its receipts.

A Numerical error bound

This appendix supplies the arithmetic details behind the conservative spectral enclosures used in Section 5.1.

The row generator and the search evaluate the cosine table in IEEE binary64 arithmetic, rounded to nearest, without fast-math transformations. The public build invokes GCC with `-O3 -fopenmp` and links `libm`; it invokes G++ with `-O3 -std=c++17 -fopenmp` for the second checker. The recorded CPU search additionally used `-march=native`. No build uses `-ffast-math`. The numerical bound assumes ordinary IEEE binary64 operations rather than x87 extended intermediate precision; a build changing the arithmetic model requires its own justification. The actual table entries are checked against interval evaluations of cosine at 50 decimal digits. Their errors are at most 2^{-49}. For order 65 the eigenvalue sum is split into two sums of at most 16 terms. Each partial sum has magnitude at most 16, so the error in one half is bounded by

$$16\,2^{-49} + 16^2\,2^{-53} < 6 \cdot 10^{-14}.$$

Combining the halves, doubling, and adding one gives eigenvalue error below $2.6 \cdot 10^{-13}$. Since $|\lambda_x(k)| \le 65$, the error in the computed square, including rounding of the multiplication, is below

$4 \cdot 10^{-11}$. Fusing a multiplication and addition does not enlarge this bound. The row filter rejects only when the computed square exceeds $260 + 10^{-6}$, so it retains every solution row.

B Storage and key implementation

This appendix explains why the implemented key and data structures preserve every pair equality required by a solution. The coefficients R_s are the successive outputs of the public `splitmix64` function, initialized with the hexadecimal seed `5713a1d2c0ffee65`, with the low bit set to one. The outputs are indexed from $s = 0$ to 32, with indices 1 through 32 used in f. The fixed multiplier R is hexadecimal `d6e8feb86659fd93`. The complete key function is specified in `checks/search.c`. Their values affect collisions and cost, but the argument needs only that both sides use the same function.

The pair bounds are formed in signed integer arithmetic before conversion to unsigned 16-bit values. The stored row values are below 16,642; all intervening sums and differences fit the signed integer type. Frequency reordering and vectorized comparisons do not change the predicates. In a vectorized upper test, a row with $\ell_x > U$ is discarded first, before forming $U - \ell_x$; the lower difference is $\max\{0, V - h_x\}$. The remaining frequencies are tested with signed integer sums. Empty lists contain no pairs and cannot yield a solution.

Surviving pairs on one side are stored by linear probing in a table whose size exceeds twice the number of entries. No entry is deleted or overwritten. A probe continues through every equal key until the first empty slot, so repeated keys and repeated rows are retained. A capacity overflow abandons the whole attempt and restarts in passes partitioned by the high bits of this same key on both sides. Equal keys always belong to the same pass. Every reported order-65 unit completed in one pass.

C Corruption controls

This appendix records deliberate alterations that the independent checkers must reject. They test whether the checks detect the particular coverage and accounting errors for which they are designed.

Corruption controls exercise both implementations: deletion of a receipt, a duplicate unit, an incorrect row-list size, an incorrect pair count, keys truncated below their prescribed 64 bits, a missing cover representative, and a false solution must all be rejected. Additional row-data corruptions test that the enumerated classes have not silently lost a row. The planted cover gap is tested by the exhaustive cover computation itself, by re-enumerating the admissible tuples.

D Positive matching control at order 65

This control tests the matching path on a full order-65 unit whose normal 64-bit keys have no matches. It truncates keys to 32 bits to create deliberate collisions, leaving the row lists and spectral filters unchanged. This is a diagnostic setting, not the setting used to conclude nonexistence.

For unit 5200, the four list sizes are 56,372, 13,092, 12,776, and 2,830. The two sides have 6,611,810 and 1,872,629 surviving pairs. The public search and a separately implemented join both find exactly 2,918 matching 32-bit keys and zero exact solutions. These matches are false collisions, not Williamson witnesses. Changing the expected match count makes the control reject. The default public check includes this test, alongside the unmodified 64-bit receipt checks.

For scale only, if the pair keys were independent uniform random 64-bit values, the expected total number of equal keys would be

$$\frac{\sum_U S_{U,1} S_{U,2}}{2^{64}} = \frac{6{,}493{,}019{,}596{,}516{,}340{,}568}{2^{64}} \approx 0.3519872976,$$

where $S_{U,1}$ and $S_{U,2}$ are the survivor counts on the two sides of unit U. At 32 bits the same hypothetical model gives about 2,883 matches for unit 5200, compared with the observed 2,918. The actual keys are deterministic and structured: no uniformity or independence is proved, and these ratios are not probability bounds or evidence of complete exhaustion. The control's evidential content is the exact agreement of the two implemented joins on a nonzero match count.

E Computation by row-sum type

The following breakdown of the CPU receipts shows how the search cost is distributed among the five possible absolute row-sum types. Survivors count both sides of each pair join; CPU times are rounded.

Absolute row sums	Units	Surviving pairs	CPU-hours
$(1, 3, 5, 15)$	19,793	439,857,850,463	55.946
$(1, 3, 9, 13)$	21,291	528,816,451,429	102.765
$(3, 3, 11, 11)$	4,733	103,766,383,699	16.208
$(3, 7, 9, 11)$	19,692	449,073,745,221	70.162
$(7, 7, 9, 9)$	5,190	124,377,127,312	26.245

F Record-checking and control details

This appendix specifies implementation sharing and explains the numerical comparison flags omitted from the main computational summary.

The record checks establish the cover and its arithmetic accounting. The first checker reuses the CPU row and unit generators to regenerate their outputs, constructs a separate exact-vector cover enumeration, computes its own count of minimal masks under K, and checks every receipt against these data. In particular, each unit appears exactly once, its listed row counts agree, and each enumerated pair count is the product of the corresponding list sizes. The checker rejects records whose option fields do not state the full 64-bit key and prescribed necessary filters, and requires each recorded solution count to be zero. It checks the recorded options; this alone does not establish that the executable honored them. Survivor and key-match counts are read from the records, not independently derived by this checker. Its cover code does not import the unit generator or the search.

The second checker was written from the mathematical specification without using the first implementation's code. It shares the problem definition and the supplied records, not the row generator, cosine table, cover-enumeration code, or receipt-checking code. It enumerates all 2^{32} masks with its own C++ code and integer cosine enclosures, obtains the same row set, performs its own exact cover enumeration, and validates all unit records. Its regenerated row file agrees byte for byte with the first implementation's. Both checkers accept the same cover and receipt totals. They do not rederive the survivor or key-match counts.

Keys truncated to 12 bits test collisions. A constrained-capacity test exercises restarts; its effective initial capacity is 1,024 entries. When a value $64\widetilde{\lambda}_x(k)^2 \pm 0.01$ is within 10^{-7} of an

integer, different summation orders may change its rounded enclosure. The automatic survivor-count comparison is skipped for that unit, while the independent solution search still runs. This affects 103 of the 312 sampled units. Comparing the printed integer counts separately shows that both survivor counts agree with their receipts in all 312 units, including those 103. At order 55, the same implementation checks 300 units sampled with seed 55 and the additional known-solution unit. It recovers the solution; the printed survivor counts agree in all 301 units, including the 29 randomly sampled units whose automatic count comparison is skipped.

G Corroborating GPU computations

This appendix distinguishes the earlier complete GPU searches from the CPU implementation distributed with this paper. Their results are corroborating computational evidence, not formal verification of the CPU program or per-unit agreement with its stronger pair filter.

Both GPU computations covered all 70,699 compressed units and found zero solutions. We call them the *primary GPU search* and the *independent GPU replay*. The primary GPU search recorded about 23.805 GPU-hours of full-search time in its portable progress summary. The independent GPU replay used a separately written implementation, regenerated the row sets and checked the complete unit cover before enumerating 146,369,334,587,912 ordered pairs. It retained 3,610,911,823,878 pairs and found 30 hash matches, all rejected by exact autocorrelation checks. Its completed search intervals contain each unit exactly once. This independent GPU replay recorded about 2.060 GPU-hours of search time and about 2.143 occupied GPU-hours including setup and controls. These resource measures are historical; they are not controlled performance comparisons between algorithms.

The independent GPU replay used an integer lower bound for the pair spectrum and did not use either the product-identity key or the one-designated-row multiplier reduction. It therefore enumerated a different number of pairs and retained more pairs than the CPU search. Both full searches agree with the CPU search on the mathematical answer, not on survivor or match totals. Their inputs, recorded build choices, completed unit intervals, and outputs were checked against the retained source and data records. Positive discovery controls at orders 25, 35, and 45 gave the known 10, 0, and 1 equivalence classes.

The earlier GPU sources and output collections are not distributed in this public CPU bundle. The portable record of the primary GPU search does not recheck all original raw execution records, and neither CUDA program is formally verified. The later CPU search is a separately coded, complete repetition of the mathematical exhaustion; the fast public record checks are not another complete repetition. All implementations share the mathematical specification, and no independent human-team or institutional replication is claimed. The self-contained reproduction route supplied here is the complete CPU search, with the trust boundary stated in Section 6.

References

[1] J. Williamson, Hadamard's determinant theorem and the sum of four squares, Duke Mathematical Journal **11** (1944), 65–81. https://doi.org/10.1215/S0012-7094-44-01108-7.

[2] R. J. Turyn, An infinite class of Williamson matrices, Journal of Combinatorial Theory, Series A **12** (1972), 319–321. https://doi.org/10.1016/0097-3165(72)90095-7.

[3] L. Baumert, S. W. Golomb and M. Hall, Jr., Discovery of an Hadamard matrix of order 92, Bulletin of the American Mathematical Society **68** (1962), 237–238. https://doi.org/10.1090/S0002-9904-1962-10761-7.

[4] L. Baumert and M. Hall, Jr., Hadamard matrices of the Williamson type, Mathematics of Computation **19** (1965), 442–447. https://doi.org/10.2307/2003674.

[5] D. Ž. Đoković, Williamson matrices of orders $4 \cdot 29$ and $4 \cdot 31$, Journal of Combinatorial Theory, Series A **59** (1992), 309–311. https://doi.org/10.1016/0097-3165(92)90071-2.

[6] D. Ž. Đoković, Note on Williamson matrices of orders 25 and 37, Journal of Combinatorial Mathematics and Combinatorial Computing **18** (1995), 171–175. https://combinatorialpress.com/jcmcc/vol18/.

[7] D. Ž. Đoković, Williamson matrices of order $4n$ for $n = 33, 35, 39$, Discrete Mathematics **115** (1993), 267–271. https://doi.org/10.1016/0012-365X(93)90495-F.

[8] W. H. Holzmann, H. Kharaghani and B. Tayfeh-Rezaie, Williamson matrices up to order 59, Designs, Codes and Cryptography **46** (2008), 343–352; a 2007 manuscript. https://doi.org/10.1007/s10623-007-9163-5.

[9] W. Lang and E. Schneider, Turyn type Williamson matrices up to order 99, Designs, Codes and Cryptography **62** (2012), 79–84; first published online March 5, 2011. https://doi.org/10.1007/s10623-011-9492-2.

[10] S. Y. Zhang, Programmable Emergence via Cognitive Engine: Longitudinal Evidence of AI Research Maturation, April 2, 2026. https://doi.org/10.5281/zenodo.19392279.

[11] D. Ž. Đoković and I. S. Kotsireas, Compression of periodic complementary sequences and applications, Designs, Codes and Cryptography **74** (2015), 365–377; arXiv:1302.0571, first posted February 4, 2013. https://doi.org/10.1007/s10623-013-9862-z.

[12] C. Bright, I. Kotsireas and V. Ganesh, Applying Computer Algebra Systems with SAT Solvers to the Williamson Conjecture, arXiv:1804.01172, first posted April 3, 2018; Journal of Symbolic Computation **100** (2020), 187–209. https://arxiv.org/abs/1804.01172. https://doi.org/10.1016/j.jsc.2019.07.024.

[13] A. Bennett, C. Bright, P. Colinot and A. Nayak, Quaternionic Perfect Sequences and Hadamard Matrices, arXiv:2601.22337, first posted January 29, 2026. https://arxiv.org/abs/2601.22337.

[14] C. Bright, Extended Research Statement, March 20, 2019. https://cs.curtisbright.com/extended-research-statement.pdf.

[15] R. Egan, I. Kotsireas, S. London and P. Ó Catháin, Recent progress in the construction of Hadamard matrices, arXiv:2610.27175, first posted October 19, 2026. https://arxiv.org/abs/2610.27175.

[16] H. Kharaghani, A. Mohammadian, B. Tayfeh-Rezaie and V. Zaitsev, A Search for Hadamard Matrices of Near-Williamson Type, arXiv:2605.08661v2, September 6, 2026. First posted May 9, 2026 as version 1 by H. Kharaghani, A. Mohammadian and B. Tayfeh-Rezaie, titled *A search for Hadamard matrices of Williamson type*. https://arxiv.org/html/2605.08661v2.